Modified DenseNet for Automatic Fabric Defect Detection With Edge Computing for Minimizing Latency

Zongwei Zhu, Guangjie Han, *Senior Member, IEEE*, Gangyong Jia, *Member, IEEE*, and Lei Shu, *Senior Member, IEEE*

***Abstract*—As an essential step in quality control, fabric defect detection plays an important role in the textile manufacturing industry. The traditional manual detection method is inaccurate and incurs a high cost; as a result, it is gradually being replaced by deep learning algorithms based on cloud computing. However, a high data transmission latency between end devices and the cloud has a significant impact on textile production efficiency. In contrast, edge computing, which provides services near end devices by deploying network, computing and storage facilities at the edge of the Internet, can effectively solve the above-mentioned problem. In this article, we propose a deep-learning-based fabric defect detection method for edge computing scenarios. First, this article modifies the structure of *DenseNet* to better suit a resource-constrained edge computing scenario. To better assess the proposed model, an optimized cross-entropy loss function is also formulated. Afterward, six feasible expansion schemes are utilized to enhance the data set according to the characteristics of various defects in fabric samples. To balance the distribution of samples, proportions of various defect types are used to determine the number of enhancements. Finally, a fabric defect detection system is established to test the performance of the optimized model used on edge devices in a real-world textile industry scenario. Experimental results demonstrate that compared with the conventional convolutional neural network (CNN), the proposed optimized model attains an average improvement of 18% in the area under the curve (AUC) metric for 11 defects. Data transmission is reduced by approximately 50% and latency is reduced by 32% in the Cambricon 1H8 platform compared with a cloud platform.**

***Index Terms*—Convolutional neural network (CNN), edge computing, fabric defect detection, image processing.**

Manuscript received December 15, 2019; revised March 4, 2020; accepted March 18, 2020. Date of publication March 24, 2020; date of current version October 9, 2020. This work was supported in part by the National Science Youth Fund of Jiangsu Province under Grant BK20190224, in part by the Jiangsu Postdoctoral Science Foundation under Grant 2019K224, in part by the National Key Research and Development Program under Grant 2017YFE0125300, in part by the National Natural Science Foundation of China–Guangdong Joint Fund under Grant U1801264, in part by the Jiangsu Key Research and Development Program under Grant BE2019648, in part by the National Natural Science Foundation of China under Grant 61602137, and in part by the Zhejiang Natural Science Foundation Project under Grant LY19F020044. *(Corresponding author: Guangjie Han.)*

Zongwei Zhu is with the Suzhou Institute for Advanced Study, University of Science and Technology of China, Suzhou 215000, China (e-mail: zzw1988@ustc.edu.cn).

Guangjie Han is with the College of Engineering, Nanjing Agricultural University, Nanjing 210095, China, and also with the Department of Information and Communication Systems, Hohai University, Changzhou 213022, China (e-mail: hanguangjie@gmail.com).

Gangyong Jia is with the Department of Computer Science, Hangzhou Dianzi University, Hangzhou 310018, China, and also with the Key Laboratory of Complex Systems Modeling and Simulation, Hangzhou Dianzi University, Hangzhou 310018, China (e-mail: gangyong@hdu.edu.cn).

Lei Shu is with the College of Engineering, Nanjing Agricultural University, Nanjing 210095, China, and also with the School of Engineering, University of Lincoln, Lincoln LN6 7TS, U.K. (e-mail: lei.shu@ieee.org).

Digital Object Identifier 10.1109/JIOT.2020.2983050

I. Introduction

During textile production, occurrences of inferior fabrics are inevitable. Therefore, defect detection is necessary and can reduce the price of fabric products by 45%–65% [1]. However, the traditional detection method that relies on manual inspection is time consuming and costly and has become the bottleneck of the entire production line. To improve detection efficiency, increasingly many studies of fabric defect detection have been carried out, mainly relying on statistical analysis [2], spectrum analysis [3], and model-based analysis [4]. Some of these methods can achieve high precision but have poor generalizability. With the development of computing hardware, methods based on deep learning have been widely explored. For example, Son *et al.* [5] proposed an efficient automatic meter reading system using the deep learning technology. The convolutional neural network (CNN) can also provide extraordinary performance optimization in the spatial image feature extraction [6]–[8]. Xu *et al.* [9] proposed an improved method using a CNN-based skip-gram method for the Chinese text classification. Yang and Min [10] presented a deep CNN-based approach for classifying artistic media using artwork images. Soukup and Huber-Mörk [11] performed experiments using a classic CNN and verify that a CNN is superior to the traditional model-based methods (e.g., Markov random field model and autoregressive model). Masci *et al.* [12] compared max-pooling CNNs with the SVM algorithm and detect seven defects in a steel strip. Experimental results show the outstanding ability of CNN in feature extraction. However, previous studies rarely explore practical problems in various industrial scenarios. Considering automated fabric defect detection as an example, noise interference in industrial environments, generalizability of models, system processing speed, and flexibility are all factors to be considered.

Fabric data collected in complex industrial scenarios often entail problems, such as dim lighting, excessive noise, and large rotation angles, which can significantly affect the

2327-4662 © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 06:12:10 UTC from IEEE Xplore. Restrictions apply.

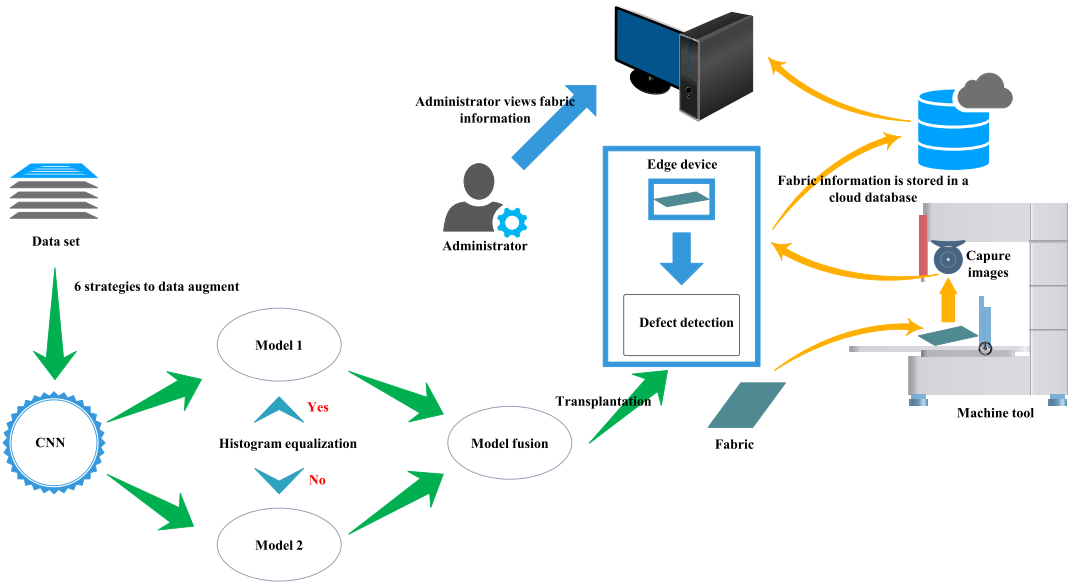


Fig. 1. Schematic of a fabric defect detection system.

recognition accuracy of a model trained using the collected data. Therefore, to obtain a robust model, the data set must be handled wisely. Studies have shown that data equalization [13] and data augmentation [14] can enhance the quality of the training data set, thus improving the quality of trained deep learning models. In this article, we first balance the training data set. Afterward, to improve the adaptability of the trained model to various complex scenarios in the production environment, the study performs a series of enhancements on the data set. For example, the random cropping strategy is applied to mimic various camera perspectives when capturing samples. The random flipping strategy is used to mitigate the sample diversity problem. A masking strategy is applied to optimize the quality of images containing dust and knots. A random brightness gain strategy is used to compensate for the effect of lighting on image quality. The random increase in noise strategy can improve the model's generalization performance. The histogram equalization (HE) strategy can create subtle differences in shadows and result in higher contrast, thus solving the problem that it is difficult to distinguish images with low contrast.

Considering that the defect detection speed in the textile production process is required to allow real-time detection, a combination of deep learning and cloud computing is inapplicable. Cloud computing methods need to transfer large amounts of data through the network, which places high demands on network bandwidth and often results in severe delays. In contrast, edge computing, with its low latency, efficient use of bandwidth, good scalability, and high confidentiality [15], has become an effective method applied in multiple scenarios, such as developing a food recognition system for diet assessment [16], an epilepsy localization system [17], etc. In edge computing, end devices usually have limited computing power and storage resources. As a result, to adapt to resource constraints of edge computing, this article uses a lightweight convolutional network called *DenseNet* [18]. At the same time, to meet the requirements of prediction speed in industrial production, the preprocessing of input sample images is omitted. Each input sample is uniformly scaled up to the size of 512*512 and transmitted to the network without trimming. Additionally, the optimized DenseNet shields the last three layers of the original network structure and adds a dropout layer, which reduces the risk of overfitting in the network training process. A global average pooling layer passing through the 11-node fully connected layer is also added. This design enables the model to detect up to 11 kinds of defects because the number of output nodes of the network is 11. The study also proposes a new loss function to solve the overconfidence problem in the cross-entropy loss function.

In an industrial scenario, automatic fabric defect detection needs to meet three requirements. First, because the detection system needs to operate synchronously with the production line running in real time, the system needs to have a low response time. Second, to minimize production costs, the system needs to have low-power consumption. Third, because various textile production lines differ from each other, and even old production lines may need to be upgraded in the future, the detection system must scale well. To satisfy the above requirements, in this article, we design an edge-cloud collaborative processing method for industrial automatic fabric defect detection. The system architecture is shown in Fig. 1. First, we need to deploy the trained lightweight model on an intelligent edge camera. Second, the latter can be flexibly positioned on the production line for real-time fabric defect detection. Finally, to effectively manage the entire defect detection, data collection, and analysis process, this article develops a management software to collect the detection results uploaded by the intelligent edge camera.

We compare data transmission and latency of the cloud computing platform and the Cambricon 1H8 platform in the experiment. The experimental results show that running on the

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 06:12:10 UTC from IEEE Xplore. Restrictions apply.

1H8 platform allows data transmission to be reduced by 50% and latency to be reduced by 32%. The main contributions of this article include the following.

1) To improve the adaptability of the fabric defect detection model in a complex industrial production environment, we use data equalization and six data enhancement methods to process the data set and compare the performance of the model before and after data processing in detail.
2) To accommodate the limitation of intelligent devices' computing resources at the edge of the network, we remove redundant network connections through network pruning. In addition, we add a dropout layer and propose a cross-entropy loss function to avoid overfitting the model.
3) This article proposes an automated fabric defect detection system. A lightweight network model is deployed on the high-efficiency Cambrian 1H8 edge smart terminal, and real-time fabric defect detection results are uploaded to the cloud management software.

All three parts constitute an industrial automatic fabric defect detection system that can be successfully applied and deployed in various industrial production scenarios.

The remainder of this article is arranged as follows. Section II introduces the design of the data set analysis architecture, the CNN, and the loss function's configuration. In Section III, various enhancement schemes for data sets are illustrated. Section IV presents a complete illustration of the hardware platform and the fabric defect detection management software and subsequently discusses the results of experiments conducted to evaluate the proposed methods and their performance on various edge computing platforms. The related studies are introduced in Section V. Finally, Section VI concludes this article.

II. Architecture Design

Deep-learning-based methods have been proven to be superior to nondeep learning methods, and common CNN structures include the visual geometry group network (VGG) [19], GoogleNet [20], ResNet [21], LeNet [22], AlexNet [23], etc. Presently, CNNs are widely used in classification and recognition tasks in the field of deep learning. One common method is to fine-tune existing networks to suit specific needs. In essence, studies aim to satisfy the following requirements.

1) The network must be capable of adapting to all kinds of noise interference in industrial production.
2) The network must be capable of extracting features from a small number of samples.
3) The network must be capable of satisfying the requirement of inspection speed for industrial production.

Considering the above requirements, we propose the following solution. First, a series of operations are performed to enhance the data set to adapt to various sources of noise in the production environment. Second, to guarantee the efficiency of the feature extraction of the network, the data set is balanced during the enhancement process. Next, the enhanced image is scaled to 512*512 to improve detection speed. Finally, the processed data are fed into the neural network for training to obtain a model that adapts to various problems. As Fig. 1 shows, CNN uses a custom loss function to enhance its learning capabilities. Depending on whether the input data set has undergone an HE enhancement operation to generate two different models, the latter are merged to obtain the final model.

TABLE I
Distribution of Positive and Negative Samples in the Data Set

	Positive	Negative	Total
Training set	706	1316	2022
Test set	462	847	1309

A. Data Set Analysis

The data set used in this article is derived from real-world fabric images extracted by a camera on an actual production line and the size of each image is 2560*1920 pixels. The data set used for training contains 2022 images and includes 46 types of defects. If a given sample contains defects, it is called *positive*; otherwise, it is called *negative*. Table I describes the numbers of positive and negative samples in the training set and the test set. Fig. 2 depicts the distribution of 46 defect types in positive samples. In Fig. 2, the abscissa describes all different kinds of defects in positive samples of the data set, while the ordinate describes the number of samples with the corresponding types of defects. The figure shows that two-thirds of defect types correspond to fewer than ten samples.

A preliminary analysis of the data set shows that it has the following characteristics.

1) The sample distribution is unbalanced, and the number of individual defect categories is extremely limited, as shown in Fig. 2.
2) The variance of the defect aspect ratio is large and some defects even run through the entire image, as demonstrated in Fig. 3.
3) Most defect areas are small, as shown in Fig. 4.
4) Most fabrics are of a single color, and gray histograms are concentrated.

As shown in Fig. 2, proportions of all types of defects in positive samples are significantly unbalanced. For example, the number of *stretched warp* defects is 125 and the number of defects in more than 30 categories is less than 20. To make each kind of defects have enough samples to enable the neural network to extract features, this article classifies *oil* and *dirt* into the *stain* category and classifies the types of defects with fewer than 20 samples as *other*. As shown in Fig. 3, because the original images are too large, the defective area is generally small. Therefore, only part of the area containing the defect is extracted and the location of the defect is indicated by the green rectangle. The ten most common defects are numbered from D1 to D10: D1 is *pin holes*, D2 is *burl mark*, D3 is *chafed yarn*, D4 is *rough*, D5 is *loose warp*, D6 is *stretched warp*, D7 is *end out*, D8 is *overshot*, D9 is *stain*, D10 is *other defect*, and D11 is *normal sample*. These labels will be used repeatedly in the remainder of this article, and the names of specific defect types will no longer be given. In Fig. 4,

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 06:12:10 UTC from IEEE Xplore. Restrictions apply.

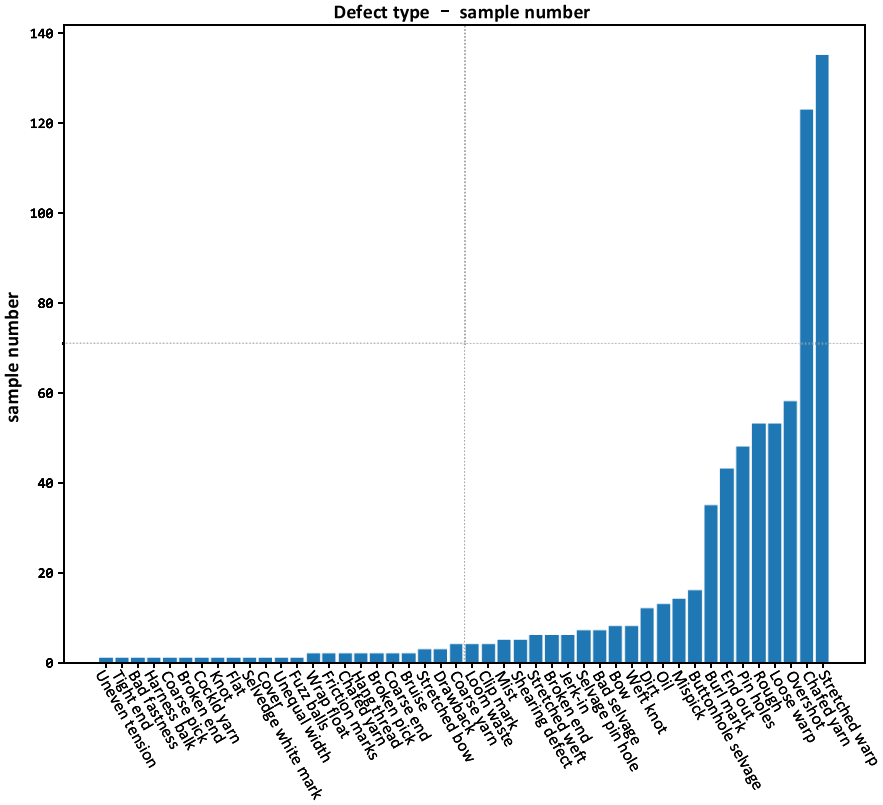


Fig. 2. Distribution of defects in the positive samples in the data set.

the abscissa shows all types of defects in positive samples of the data set, while the ordinate represents the size of the defect area of the corresponding type of defects. The blue bar represents the maximum size of the defect; the orange bar represents the average size; and the green bar represents the minimum size.

A preliminary classification results in 11 categories, including the *normal* type; the distribution of each type of samples in the data set can be found in the *origin* row of Table II. It is observed that the proportions of defect samples of various types are somewhat balanced, but this is still insufficient. The maximum number of *stretched warp* defects is 125, and the minimum number of *stain* defects is only 26, while the learning of a neural network tends to focus on data that are balanced in the training set. For this reason, this article augments the training data set in combination with its characteristics, which makes the proportions of 11 kinds of samples tend toward 1:1:1:1:1:1:1:1:1:1:10. Thus, the ratios of various types of defect samples reaches 1:1 and the ratio of all defect samples to normal samples is also 1:1. The balancing method is mainly used to facilitate different amounts of amplification for different types of defect samples. For example, in the *origin* row of Table II, the number of *pin hole* defects is 44. This sample is expected to be preamplified to 120 samples, and it is necessary to amplify the number of samples of the *pin hole* type by a factor of 2.7. The specific method is to first duplicate the sample and then create another copy of it with a probability of 0.7. The *after* row of Table II records the distribution of samples of each type in the data set after data amplification.

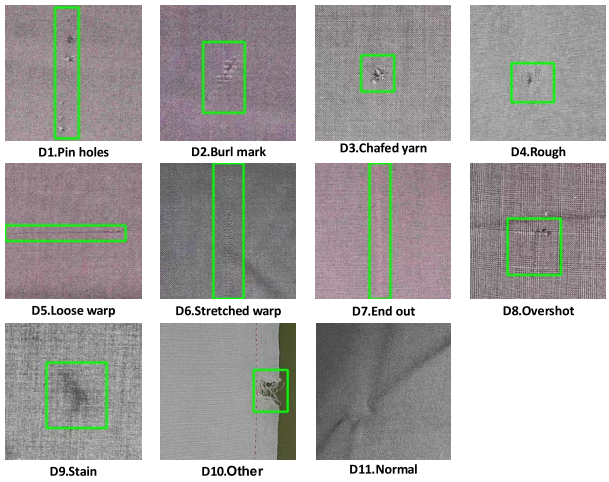


Fig. 3. Samples of several common defect types.

B. Network Architecture

DenseNet was first proposed by Gao *et al.* [18] in 2017. Due to its excellent performance, it has even been applied to problems, such as text detection [24] and audio source separation [25]. The main contribution of *DenseNet* is to fully connect all blocks, making the transmission of information and gradients in the network more efficient so that deeper networks can be trained. Compared to other networks dedicated to improving network performance from the perspectives of depth and breadth, *DenseNet* improves network performance by feature reuse, which substantially improves the network's prediction speed. Based on the above-mentioned reasons,

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 06:12:10 UTC from IEEE Xplore. Restrictions apply.

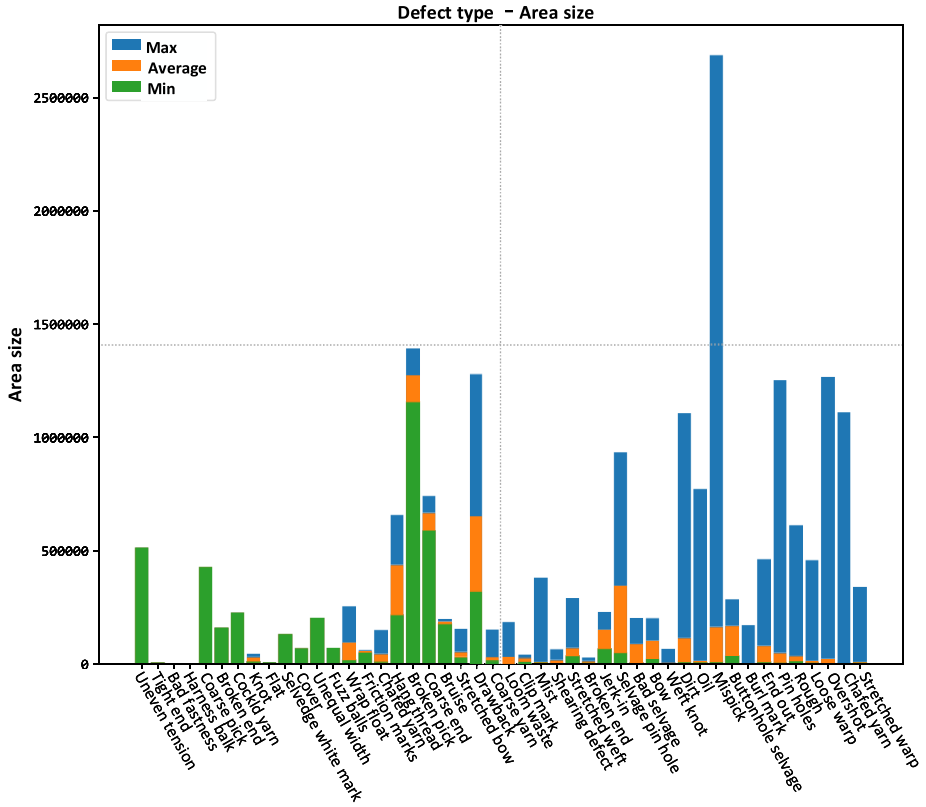


Fig. 4. Various types of defects in positive samples in the data set.

TABLE II
SAMPLING DISTRIBUTION OF EACH TYPE OF DEFECTS AFTER THE AMPLIFICATION OF THE TRAINING SET

	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10	D11
Origin	44	32	111	48	51	125	38	50	26	116	1185
Times	2.7	3.7	1	2.5	2.3	1	3.1	2.4	4.6	1	1
After	120	120	111	122	112	125	117	124	118	116	1185

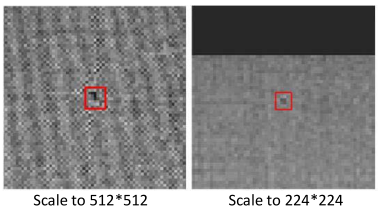


Fig. 5. Same *rough* defect area under two different scaling sizes.

and taking into account the requirements of detection speed in industrial production, we use *DenseNet* as our network architecture.

In addition, since the original image size of the training set is 2560*1920 pixels and the proportion of defective regions in each sample is generally low, scaling a defective sample's size to the default input size of 224*224 of *DenseNet* may even cause the defective region to be scaled to only one pixel, which is not conducive to extracting features in the network. If we choose a window of size 224*224 and use 112 steps for slide detection, which requires 414 times to obtain a prediction for an image, the prediction speed will be significantly affected. Generally, in training CNNs, the input sizes are 128*128, 224*224, 512*512, etc. To allow the model to fully grasp the contour information of defects, this article adjusts the input size to 512*512, which ensures the learning ability of the model while controlling the excessive amount of parameters as much as possible.

Finally, as *DenseNet* by default outputs 1000 categories, which does not meet our requirement of classifying 11 defect types, the following modifications are performed.

1) Shield the last three layers of the original network and replace them with new layers that adapt to the requirements.
2) Add a *dropout* layer to reduce the risk of overfitting during network training.
3) Add a *global average pooling* layer and finally pass through the 11-node *fully connected* layer and a *softmax* layer.

Fig. 5 shows the size of the same *rough* defect area under scaling sizes of 512*512 and 224*224. We clearly observe that there are 36 pixels in the defective area if the original image is scaled to 512*512, while there are only four pixels in the defective area if the original image is scaled to 224*224. This phenomenon further explains why it is more appropriate to set the input size of the network to 512*512. In addition, the model no longer outputs 1000 categories but is instead modified to output 11 categories that need to be predicted. The specific structure is shown in Fig. 6, where the abbreviations

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 06:12:10 UTC from IEEE Xplore. Restrictions apply.

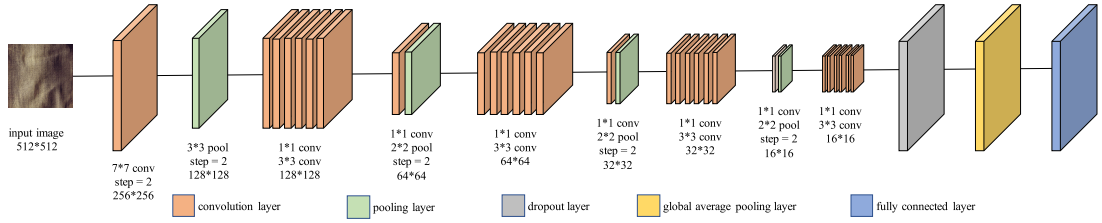


Fig. 6. Structure of the modified *DenseNet*.

conv, *pool*, and *FC* represent convolution, pooling, and the fully connected layers, respectively. The first layer outputs an image of the size of 256*256 with a convolution kernel of size 7*7, and the second layer outputs an image of size 128*128 with a window of 3*3; their step sizes are equal to 2. The third layer outputs an image of size 128*128 with two convolution kernels of sizes 1*1 and 3*3, respectively, and each kernel is executed six times. The following layers are similar to the above. The classification score is the confidence of the network's prediction after processing by the softmax layer.

C. Loss Function's Configuration

In a traditional neural network designed for classification, the loss function used during training typically has the form of cross-entropy [26]. The latter is mainly used to measure the difference between two probability distributions. Cross-entropy is computed as follows:

$$\text{loss} = -\sum_{i=1}^{n} q_i \log p_i \tag{1}$$

where p_i is the predicted distribution and q_i is the true distribution. However, cross-entropy as a loss function also has many shortcomings, one of which is that the resulting classification is too confident. The result of classification is always either 1 or 0, which often risks overfitting; additionally, in practical applications, it is impossible to determine the confidence interval and set the threshold well. Therefore, to solve the problem of overconfidence in the cross-entropy loss function, the solution adopted in this article is to not simply fit the one-hot distribution but to fit a uniform distribution with little effort. This solution results in the following new loss function:

$$\text{loss} = -\sum_{i=1}^{n} \left[\lambda \frac{1}{n} + (1-\lambda) q_i \right] \log p_i. \tag{2}$$

During the experiment, the value of λ was set to 0.25 according to our experience. After this treatment, even if the neural network blindly increases the proportion so that p_i is close to 1, the total loss function's value is no longer the optimal solution, which can mitigate the case of being too confident in the output. The experiments show that this kind of processing can significantly improve the accuracy of network prediction. The specific data will be presented in the discussion of experimental results in Section IV.

III. Data Enhancements

We can conclude from Section II that it is extremely difficult to enable neural networks to learn from a few flawed samples and detect them, which is especially challenging (to the point of being impossible) in extreme cases. Therefore, data enhancement is proposed for balancing the number of defect samples of various types. Combined with the analysis of an actual production environment and defect characteristics, we derive the following principles that will serve as a basis for data enhancement.

1) The sample image resolution is high.
2) The defect area sizes have a large variance.
3) The defect contour is rotation invariant.
4) Most fabrics are of a single color, and the gray histograms are concentrated.
5) Many fabrics exhibit nonflaw interference, such as creases and worktable backgrounds.
6) The distance between the camera and the fabric in the production process may not be consistent.
7) The brightness of the workshop may vary during the production process.
8) There may be dust, knots, and other noise in the production process that interferes with the camera photographing the fabric.

In an enhancement loop, each enhancement scheme is triggered with a 50% probability and if none of the schemes are activated, another loop is required until at least one scheme has been triggered. In other words, each picture is enhanced by at least one of the following schemes.

A. Random Clipping

Combining the first, second, and sixth principles mentioned above, an image can be randomly clipped and then scaled, which allows the neural network to learn the features of different sizes similar to the process of downsampling the image by a feature pyramid. Moreover, this can also facilitate making predictions for image samples obtained at different camera heights and enhance the robustness of the training model of the neural network.

Random clipping is a process of cropping the original image without destroying the original flaw. It requires that the defect location be determined first; afterward, the defect frame is randomly expanded without exceeding the boundaries of the largest image. As shown in Fig. 7, the trimming frame must be between the white frame and the black frame. The green frame in the image is the specific location of each flaw marked in the XML file; the white frame is the smallest border that

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 06:12:10 UTC from IEEE Xplore. Restrictions apply.

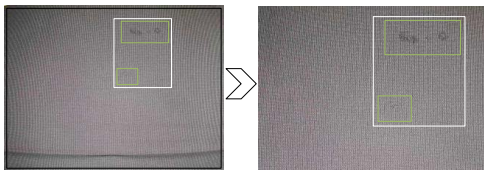


Fig. 7. Schematic of random clipping requiring that the clipping box be between white and black boxes.

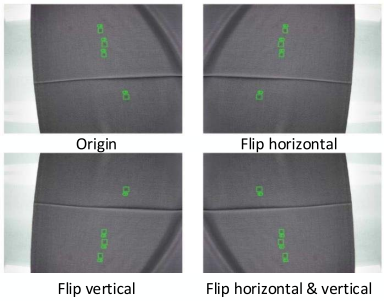


Fig. 8. Diagrams of random flipping.

can contain all flaws; and the black frame is the border of the original image.

B. Random Flipping

Rotation invariance in mathematics refers to a function defined in the inner product space. If the parameter value of the function changes for any rotation but the value of the function remains unchanged, then the function is called rotation invariant or having rotational symmetry. In addition, defects that are the object of this article are also rotation invariant. Since a defect has no direction, it can still be considered the same defect after being reversed or symmetrically mapped. Based on this feature, a random flipping strategy is proposed to enhance the data set.

A random flip operation performs various flips of the image, e.g., horizontal flips, vertical flips, and horizontal and vertical flips, and at least one flip condition is required in the enhancement process. A stochastic flip diagram is shown in Fig. 8.

C. Histogram Equalization

The previous analysis of characteristics of data sets shows that most of the fabrics are of a single color and the gray histograms are concentrated. Moreover, observing the samples suggests that the defect type is considered unrelated to the color; therefore, the HE method is used to cause the model to pay more attention to the contour information of defects rather than to color. The gray histogram is a function of the gray-level distribution and is a statistic of the gray-level distribution in the image. It is used to show the frequency of all pixels in a digital image according to the magnitude of the gray value. A grayscale histogram is a function of the grayscale, which represents the number of pixels with a certain gray level and reflects the frequency of occurrence of a certain gray level in the image. The gray histogram is computed according to the

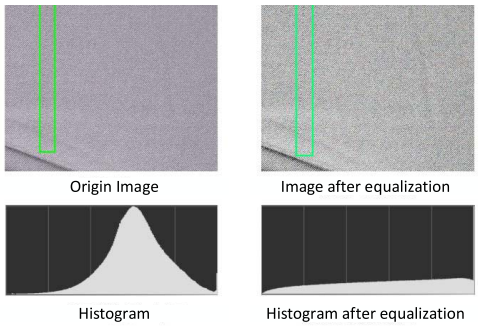


Fig. 9. Image's gray HE.

following formula:

$$P(S_k) = \frac{n_k}{n} \tag{3}$$

where S_k is the gray level of the pixel, ranging from 0 to 255, with 0 representing pure black and 255 representing pure white. Variable n_k denotes the number of pixels with a gray level of S_k, and n is the total number of pixels in the image. Taking S_k as the abscissa and $P(S_k)$ as the ordinate in the Cartesian coordinate system, the gray histogram of an image can be obtained.

If the grayscale distribution of an image is not uniform, it is concentrated within a narrow range, the details of the image are not sufficiently clear, and the contrast is low. Usually, HE and histogram normalization are used to make the grayscale range of an image open or to evenly distribute the grayscale, thereby increasing the contrast, making the image details clear, and achieving enhancement.

HE stretches an image nonlinearly and redistributes the gray values of the image to make those values approximately equal within a certain range. Thus, the contrast of the peak portion in the middle of the original histogram is enhanced, while the contrast of the valley portions on both sides is lowered, and the histogram of the output image is relatively flat.

The specific steps of HE are as follows.

1) Calculate the gray histogram $P(S_k)$.
2) Calculate the cumulative histogram

$$\mathrm{CDF}(S_k) = \sum_{i=0}^{k} \frac{n_i}{n} = \sum_{i=0}^{k} P_s(S_i). \tag{4}$$

3) Obtain the histogram after equalization

$$P'(S_k) = \mathrm{round}\left(\frac{\mathrm{CDF}(S_k) - \mathrm{CDF}_{\min}}{n - \mathrm{CDF}_{\min}}\right) * (L-1). \tag{5}$$

In (5), $\mathrm{CDF}_{\min}$ is the minimum value of the cumulative distribution function and L is the number of gray levels; the latter in this article is 256. Fig. 9 shows the differences in histograms before and after HE.

D. Adding Auxiliary Mask

The analysis of samples shows that many fabrics have nondefective features, such as creases and workbench backgrounds, as shown in Fig. 10. The red box indicates nondefective interference and the green box is the true defect

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 06:12:10 UTC from IEEE Xplore. Restrictions apply.

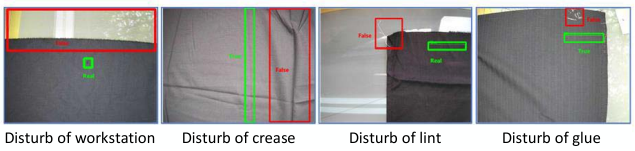


Fig. 10. Several common examples of nondefect interference.

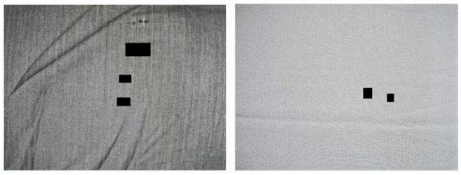


Fig. 11. Schematic of an addition of a mask.

location. Such difficult-to-classify features are easily misjudged by the neural network, resulting in an excessive false positive rate (FPR).

To adapt to such interference and make the neural network more focused on the extraction of defect features, this article proposes adding masks. Adding some pure black areas to the image causes the defective parts to be *covered* with pure black blocks, making the areas *invisible* to the neural network so that it will pay more attention to other parts of the image. In the enhancement process, some defects are randomly selected from an image for mask processing and if all defects have been masked, they are regarded as normal samples. To equalize the masked data, normal samples also need to be processed. For normal samples, some defect frames in defective samples are *mapped* to normal samples and treated as defective samples; the final results are all normal samples. This process is very effective in preventing *creases* from being misjudged as defects; the specific effects are described in detail in the discussion of experimental results in Section IV. Fig. 11 shows a schematic illustration of the addition of a mask.

E. Random Brightness Gain and Random Added Noise

In the real-world production workshops, the brightness of illumination of different workshops is not exactly the same, and the sensitivity of different types of cameras also varies. These properties can result in an uneven brightness of samples: some may be brighter, while others may be darker. Making the model adapt to these changes in brightness allows the type of defects to be correctly identified even if brightness varies. This article proposes a strategy of random brightness gain that not only enhances the prediction of the network model at different values of image brightness but also generates more different data and strengthens the learning ability of the neural network. The main approach being used is to randomly adjust the gamma value of brightness in an image by a factor of between 0.5 and 1.5 times.

In addition, the clarity of images captured by cameras with different specifications also varies, and the generated photographs exhibit some noise. Such noise is mainly caused by electronic interference, which is unavoidable and produces different amounts of noise each time. To this end, this article also proposes a strategy to randomly add noise, i.e., to enhance the performance of the model in complex production environments by randomly adding Gaussian noise to the image.

TABLE III
IMPROVEMENT OF THE MODEL'S AUC SCORE BY THE ICE

	D1	D2	D3	D4	D5	D6
Before	0.82	0.81	0.75	0.63	0.65	0.60
ICE	0.62	0.64	0.78	0.74	0.71	0.75
	D7	D8	D9	D10	D11	Average
Before	0.59	0.52	0.55	0.60	0.67	0.65
ICE	0.69	0.53	0.51	0.65	0.84	0.68

IV. EXPERIMENTAL RESULTS

The concept of a receiver operating characteristic (ROC) curve was first proposed by Emmerich [27] in 1968 and is mainly used to evaluate the quality of a model. As a very useful visual tool, it is widely used in many fields of research [28], [29]. The abscissa of the ROC curve is the FPR, and the ordinate is the true positive rate (TPR). The area under the curve (AUC) is the area under the ROC curve and has values between 0 and 1. It is a numeric value that can be used to intuitively evaluate the quality of a classifier: the larger the value is, the better the classifier. In this article, the classification ability of the model described in the experimental results is assessed based on the AUC index. The data set used in this article is from the Alibaba Tianchi Competition and was sampled from a real-world production line.

A. Experiment on the Improved Model

Table III shows that the improved loss function (ICE) introduced in Section II-C also improves the predictive power of the network model, especially for the *normal* type of sample classification, for which the improvement in the AUC index reaches 17%. This function clearly increases the recall rate of the full model and reduces the FPR. Overall, the ICE contributes a 3% average improvement to the network model.

Table IV shows that the model's predictive power improves significantly after the data balancing (DB) and data enhancing (DE) operations described in Section II-A. Among 11 categories predicted in this article, the network model improves the classification ability for almost every type of defect. Among them, the most apparent improvement is observed for the *rough* type, which is improved by 26%, and the average classification ability of 11 kinds of defects also increases by 10%. It can be concluded that the DB operation has a notable effect on the learning ability of CNNs.

Table V demonstrates that HE has a positive effect on identifying several types of defects, including clear instances of *burl mark*, *chafed yarn*, *stretched warp*, *end out*, and *other* defects. However, for some defect types, such as *rough*, *loose warp*, and *overshot*, the predictive ability declines. Merging the network models trained on data sets with or without HE shows that network model fusion (MF) improves the prediction

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 06:12:10 UTC from IEEE Xplore. Restrictions apply.

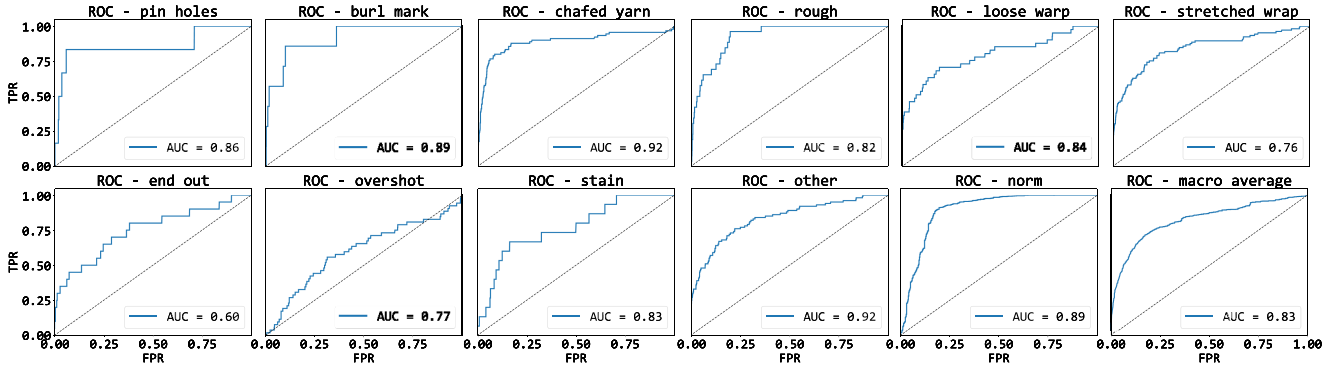


Fig. 12. Ability of the proposed approach to detect various types of defects.

TABLE IV
IMPROVEMENT IN THE MODEL'S AUC SCORE AFTER THE DB AND DE OPERATIONS

	D1	D2	D3	D4	D5	D6
Before	0.62	0.64	0.78	0.74	0.71	0.75
DB&DE	0.77	0.88	0.84	0.90	0.79	0.75
	D7	D8	D9	D10	D11	Average
Before	0.69	0.53	0.51	0.65	0.84	0.68
DB&DE	0.67	0.59	0.76	0.75	0.87	0.78

TABLE V
IMPROVEMENT IN THE AUC SCORE DUE TO HE AND MF

	D1	D2	D3	D4	D5	D6
Before HE	0.77	0.88	0.84	0.90	0.79	0.75
After HE	0.77	0.90	0.86	0.82	0.67	0.82
After MF	0.86	0.92	0.89	0.92	0.82	0.84
	D7	D8	D9	D10	D11	Average
Before HE	0.67	0.59	0.76	0.75	0.87	0.78
After HE	0.70	0.54	0.56	0.78	0.86	0.75
After MF	0.76	0.60	0.77	0.83	0.89	0.83

TABLE VI
DETECTION CAPABILITY COMPARISON OF THE FINAL IMPROVED MODEL WITH THE ORIGINAL MODEL

	D1	D2	D3	D4	D5	D6
Before	0.82	0.81	0.75	0.63	0.65	0.60
Final Improved	0.86	0.92	0.89	0.92	0.82	0.84
	D7	D8	D9	D10	D11	Average
Before	0.59	0.52	0.55	0.60	0.67	0.65
Final Improved	0.76	0.60	0.77	0.83	0.89	0.83

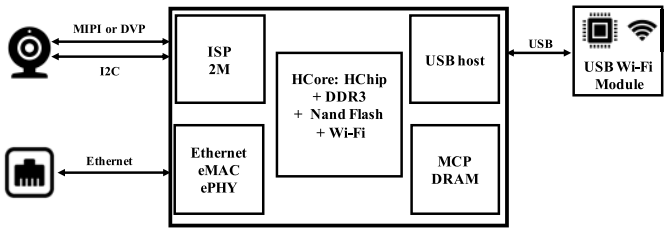


Fig. 13. Overall architecture of the hardware platform.

ability for all types of defects. Overall, HE leads to 5% average improvement in the predictive power of the network model.

In summary, the proposed schemes, including DB, data enhancement, and loss function improvement, significantly improve the predictive ability of network models for various types of defects. Fig. 12 depicts the ultimate defect detection capability of our approach in the form of ROC curves. In Fig. 12, the abscissa of the chart is the FPR, i.e., the likelihood that an incorrect example is classified correctly. The ordinate is the TPR, i.e., the likelihood that a correct example is classified incorrectly. The curves are the ROC curves corresponding to each defect type. The corresponding AUC value is shown in the lower right corner of each chart. Table VI displays a comparison of the final predictive ability with that of the original model. As the table shows, the classification ability of all types of defects has been significantly improved. The category with the weakest classification ability is *pin holes*, scoring 4%; the type with the strongest classification ability is *rough*, which reaches 29%; the classification ability increases by 18% on average, and the effect is quite remarkable.

Our method can detect more types of flaws than can other nondeep learning methods. Our proposed approach can detect ten most common kinds of flaws and even some uncommon "other" flaws. In contrast to the commonly used sliding-window detection method, this article uses a nonsegmented prediction method to greatly reduce the number of calculations and shorten the prediction time; the proposed method is a better fit for industrial production needs. Although many types of defects can be identified in this article, Fig. 12 shows that the network model does not perform as well when identifying the "*overshot*" defect, and has a weak predictive ability for this type of defect. The follow-up analysis should separately consider the *overshot* type of defect and design a prediction scheme to improve the prediction ability of the network model for 11 defect types proposed in this article.

B. Performance Comparison Experiment on Edge Device

Considering three requirements of a real-time usage scenario—low transmission latency, high energy efficiency, and excellent scalability—this article presents a design of a hardware system based on the Cambrian 1H8 platform for deployment of our industrial automatic fabric defect detection model. This hardware platform adopts the core board and bottom plate design framework. As Fig. 13 shows, the core board (HCore) in the center consists of an HChip, DDR3 memory, external NAND *flash* memory, and a Wi-Fi chip. HChip is

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 06:12:10 UTC from IEEE Xplore. Restrictions apply.

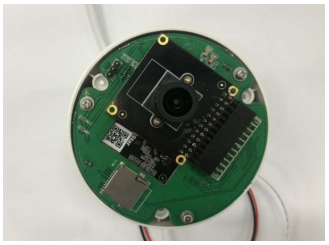



Fig. 14. Actual structure of our customized intelligent edge camera.

an integrated processor that contains a Cambrian 1H8 intelligent terminal processor IP, an ARM Cortex A7 CPU, and a video codec processor. The Cambrian 1H8 intelligent terminal processor IP is an artificial intelligence processor used in embedded terminals, with the main frequency of up to 1 GHz and the maximum power consumption not exceeding 5 W. It can perform the inference calculation of a complex deep learning network model and its integer computing capability can reach 0.8 Tops. An implementation using this processor results in a significantly improved energy efficiency ratio. The bottom plane contains several common peripheral interfaces, such as a camera, audio, USB, SD, Ethernet, SPI, I2C, UART, GPIO, and PWM. These external interfaces can be quickly customized according to various application scenarios to dramatically enhance system scalability. Fig. 14 shows the actual structure of our customized intelligent edge camera.

In addition, this article develops fabric defect detection management software to manage the entire detection process and analyze the data collected in real time. Fig. 15 shows a screenshot of the management software. As indicated by red boxes, the *detection* page lists the fabric defect detection results belonging to a specific batch and detected by a specific device. To make it convenient for users to obtain the results according to device or batch number, the respective search functions are provided. The *detail* link allows the user to switch to another page displaying the defect images. In addition, the management software also has other pages, such as *order*, *statistics*, and *maintenance*, that cover all aspects of industrial production.

The three components—the CNN detection model, the hardware platform using Cambricon 1H8, and the fabric defect detection management software—together constitute our industrial automatic fabric defect detection system based on the edge computing approach that is a complete system that can be deployed in various detection scenarios.

To verify edge computing's advantages of low data transmission and low latency, this article performs detection using a fabric image separately in the Cambricon 1H8 platform and the cloud computing platform and records the amount of data transmitted and time spent. Fig. 16 shows the sequential difference between the two platforms.

With cloud computing, detecting an image typically entails five steps: 1) capturing the image; 2) uploading it; 3) processing it in the cloud; 4) downloading the detection image and the result; and 5) the fabric detection machine takes a control action according to the result. It should be noted that the cloud host hardware is configured as Intel Xeon CPU E5-2609 v4@1.70 GHz, 64-GB RAM and 500-GB SSD, and the network test environment is built as 200M bandwidth. A fabric detection RESTful API service is deployed on the *Kubernetes* cloud operating system.

TABLE VII
RESPONSE TIME OF EDGE COMPUTING AND CLOUD COMPUTING

	Edge computing	**Cloud Computing**
Step1	Capturing images	Capturing images
	40ms	40ms
Step2	Edge computing	Uploading original images
	282ms	213ms
Step3	Machine response	Cloud computing
	52ms	48ms
Step4	Uploading detected images	Result returned
	213ms	199ms
Step5	Result returned (no images)	Machine response
	158ms	52ms
Response cycle	374ms	552ms

In edge computing, the fabric detection machine with a 1H8 intelligent edge camera has the ability to process images; therefore, there is no need to download the result. After capturing and processing the image, the fabric detection machine takes a control action and subsequently uploads the detection image (context requests) to the cloud, and the result returned.

As the gray part of Table VII shown, the process of capturing the image to the fabric detection machine response is defined as a response cycle. With cloud computing, capturing the image spends 40 ms. The uploading process takes 213 ms, it includes packaging detection image into JavaScript Object Notation (JSON) format data and network delay. The processing time is 48 ms, it contains API route analysis, image detection, and result output. The time of downloading the detection image and the result consumes 199 ms, this process includes network transmission and terminal data analysis. Finally, the fabric detection machine spends 52 ms on receiving a command via an *RS232* serial port to take a control action. Therefore, a response cycle of cloud computing is 552 ms. In contrast, if the edge computing is used, network I/O time is significantly reduced, the response cycle only includes capturing the image (40 ms), edge processing (282 ms), and machine response (52 ms). Thus, the detection cycle only needs 374 ms. In summary, compared with cloud computing, the edge computing solution has reduced the latency by 32%. Note that the above experiments focused on identifying the differences between the two computing modes. The impact of network infrastructure instability is beyond the scope of this article. In terms of data transmission, regardless of whether cloud computing or edge computing is being used, statistical analysis software that installs the fabric defect detection management software should be deployed if statistical information is required. However, this step is inessential and hence is not considered. Therefore, each image is transmitted twice over the network if cloud computing is used, yet it is transmitted half as many times if edge computing is used. Regardless of minor factors, data transmission can be reduced by 50% with edge computing.

V. RELATED WORK

In the development of industrial automation, visual defect detection efficiency has a significant impact on production

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 06:12:10 UTC from IEEE Xplore. Restrictions apply.

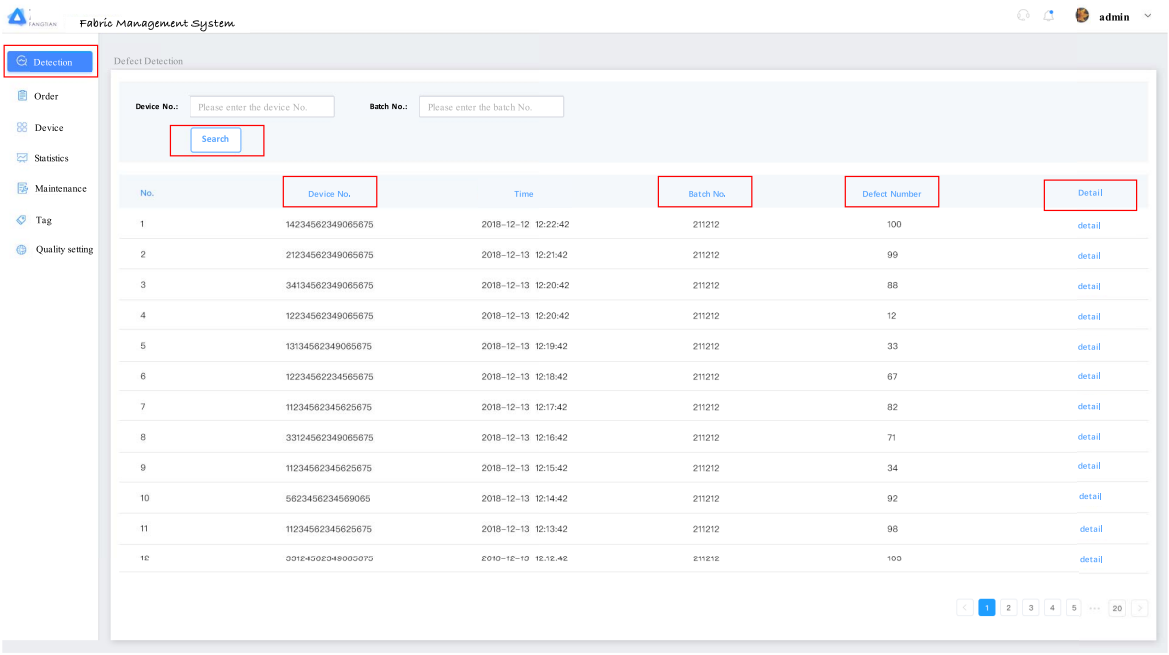


Fig. 15. Fabric defect detection management software.

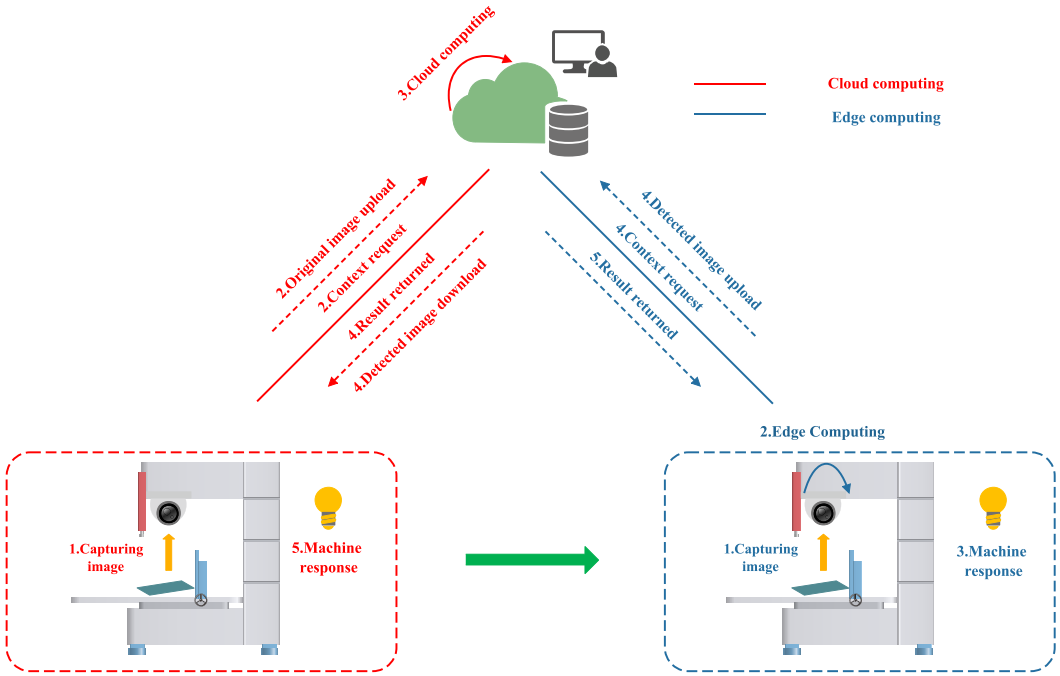


Fig. 16. Sequence diagram of cloud computing and edge computing.

efficiency and quality. Many scholars have performed various in-depth studies of machine vision technology applied to surface defects and proposed multiple methods. Traditional defect detection methods mainly include statistical analysis, frequency spectrum analysis, and model-based analysis. Raheja *et al.* [30] extracted the texture features of a fabric image based on the gray level co-occurrence matrix and deployed the respective method on an embedded platform. Such a statistical analysis-based method can only identify a limited number of defects. The method based on spectrum analysis [31], [32] is mainly used to transform the fabric image from the spatial domain to the frequency domain. Defects appear as abnormal textures and their spectral distribution must be different from that of normal textures. However, in fabric production engineering, many repeating structures with small deviations are in fact not defects, so this method has a large FPR. The method based on model analysis [33] controls the model distribution of parameters, and model parameters are used as texture features that are only applicable to relatively fixed scenes. A common shortcoming of these traditional methods is a poor model generalization performance. Additionally, the model cannot adapt to

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 06:12:10 UTC from IEEE Xplore. Restrictions apply.

various influences caused by the environment, such as lighting, production process, and noise.

To overcome the limitations of traditional machine vision-based defect detection methods, researchers have explored better defect detection strategies based on deep learning methods. A method based on deep learning mainly uses CNN to extract defect features and perform classification. The CNN obtains multilayer features of complex texture images through multiple nonlinear transformations. Therefore, compared to traditional methods, using CNN avoids complex feature selection and manual extraction. For example, Lin *et al.* [34] used a single-shot multibox detector to extract image features, and subsequently used a deep residual network to classify defects on a steel surface; Jing *et al.* [35] proposed a detection method for automatic fabric defect detection using a deep CNN, and Lin *et al.* [36] applied a CNN to LED chip defect inspection. However, deep-learning-based methods have several drawbacks. For example, model effectiveness is closely related to the quality of the data set, and the system deployment requires substantial computing resources. The above shortcomings limit the deployment of deep learning models in practical industrial scenarios.

Data augmentation is a very powerful method of building useful deep learning models [14]. For example, a classic single-stage face detection algorithm called S3FD [37] entails a sample enhancement strategy using random cropping of images, fixed-aspect-ratio scaling of images, image color perturbation, and horizontal flipping. Perez and Wang [38] used a variety of traditional data enhancement strategies (such as cropping, rotating, and flipping input images) on a small set of ImageNet data sets and proved the effectiveness of this strategy for classification tasks. In the above study, the preprocessing of the fabric data set before model training was very important.

In addition, developing energy-efficient and flexible deployment schemes is also a key issue for deep learning models' implementation [39], [40]. As a key driver that boosts AI development, big data have recently gone through a radical shift of the data sources from mega-scale cloud datacenters to the increasingly widespread end devices [41]. Edge intelligence is a key enabler of IoT, as it offers smart services in close proximity to the production environment with low latency and reduced cost [42]. Li *et al.* [43] proposed Edgent, a framework that leveraged edge computing for DNN collaborative inference through device-edge synergy. Liu *et al.* [44] proposed an intelligent monitoring method of target tracking. With the impressive development of intelligent embedded chips, more and more artificial intelligence-based application solutions are being deployed on network edges [45], [46], which provides a solid foundation for us to propose an edge-cloud collaborative fabric defect detection automation processing solution.

VI. Conclusion

The main contribution of this article is the proposed industrial automated fabric defect detection system. It uses a modified CNN model combined with our newly designed hardware platform based on edge computing that can meet several requirements of the detection task in real-time scenarios: a low response time, a high energy efficiency, and high scalability. In addition, six data enhancement schemes and improved cross-entropy loss functions are proposed to improve the ability of the CNN model to predict fabric defects. To evaluate the proposed scheme, extensive experiments are performed on a fabric defect data set that consists of different defect types and marking information. The experimental results demonstrate that compared with a conventional CNN, the optimized DenseNet model attains on 11 defects an average improvement of 18% in the AUC metric. Data transmission is reduced by approximately 50%, and the latency is reduced by 32% in the Cambricon 1H8 platform compared with the cloud platform. Our approach can significantly improve the learning ability of a CNN. The trained network model both performs detection rapidly and is very robust. Finally, to improve production efficiency, this article has developed fabric defect detection management software to manage the entire inspection process.

References

[1] K. Srinivasana, P. H. Dastoora, P. Radhakrishnaiaha, and S. Jayaraman, "FDAS: A knowledge-based framework for analysis of defects in woven textile structures," *J. Textile Inst.*, vol. 83, no. 3, pp. 431–448, Jan. 1992.

[2] F. S. Najafabadi and H. Pourghassem, "Corner defect detection based on dot product in ceramic tile images," in *Proc. IEEE Int. Colloquium Signal Process. Appl.*, 2011, pp. 293–297.

[3] H. Elham, F. Farnoush, and T. Farshad, "Fabric defect detection using auto-correlation function," *Int. J. Comput. Theory Eng.*, vol. 5, no. 1, pp. 114–117, 2013.

[4] M. Ghazvini, S. A. Monadjemi, N. Movahhedinia, and K. Jamshidi, "Defect detection of tiles using 2D-wavelet transform and statistical features," in *Proc. World Acad. Sci. Eng. Technol.*, vol. 49, 2009, p. 901.

[5] C. Son, S. Park, J. Lee, and J. Paik, "Deep learning–based number detection and recognition for gas meter reading," *IEIE Trans. Smart Process. Comput.*, vol. 8, no. 5, pp. 367–372, 2019.

[6] H. R. Roth *et al.*, "Anatomy-specific classification of medical images using deep convolutional nets," in *Proc. IEEE 12th Int. Symp. Biomed. Imag. (ISBI)*, Apr. 2015, pp. 101–104.

[7] T. Guo, J. Dong, H. Li, and Y. Gao, "Simple convolutional neural network on image classification," in *Proc. IEEE 2nd Int. Conf. Big Data Anal. (ICBDA)*, Mar. 2017, pp. 721–724.

[8] R. C. Çalik and M. F. Demirci, "Cifar-10 image classification with convolutional neural networks for embedded systems," in *Proc. IEEE/ACS 15th Int. Conf. Comput. Syst. Appl. (AICCSA)*, Oct. 2018, pp. 1–2.

[9] W. Xu, H. Huang, J. Zhang, H. Gu, J. Yang, and G. Gui, "CNN-based skip-gram method for improving classification accuracy of chinese text," *KSII Trans. Internet Inf. Syst.*, vol. 13, no. 12, pp. 6080–6096, 2019.

[10] H. Yang and K. Min, "A deep approach for classifying artistic media from artworks," *KSII Trans. Internet Inf. Syst.*, vol. 13, no. 5, pp. 2558–2573, 2019.

[11] D. Soukup and R. Huber-Mörk, "Convolutional neural networks for steel surface defect detection from photometric stereo images," in *Proc. Int. Symp. Vis. Comput.*, vol. 8887, 2014, pp. 668–677. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-319-14249-4_64

[12] J. Masci, U. Meier, D. Ciresan, J. Schmidhuber, and G. Fricout, "Steel defect classification with max-pooling convolutional neural networks," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2012, pp. 1–6.

[13] K. Oksuz, B. C. Cam, S. Kalkan, and E. Akbas, "Imbalance problems in object detection: A review," 2019. [Online]. Available: https://arxiv.org/abs/1909.00169

[14] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *J. Big Data*, vol. 6, no. 1, p. 60, 2019.

[15] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, 2016.

[16] C. Liu *et al.*, "A new deep learning-based food recognition system for dietary assessment on an edge computing service infrastructure," *IEEE Trans. Services Comput.*, vol. 11, no. 2, pp. 249–261, Mar./Apr. 2017.

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 06:12:10 UTC from IEEE Xplore. Restrictions apply.

[17] M. P. Hosseini, T. X. Tran, D. Pompili, K. Elisevich, and H. Soltanian-Zadeh, "Deep learning with edge computing for localization of epileptogenicity using multimodal RS-FMRI and EEG big data," in *Proc. IEEE Int. Conf. Auton. Comput. (ICAC)*, 2017, pp. 83–92.

[18] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," 2016. "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 2261–2269.

[19] K. Simonyan and A. Zisserman, "Very deep revolutionary networks for large-scale image recognition," 2014. [Online]. Available: arXiv:1409.1556.

[20] C. Szegedy *et al.*, "Going deeper with convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 1–9.

[21] H. Kaiming, Z. Xiangyu, R. Shaoqing, and S. Jian, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

[22] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[23] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105.

[24] M. Behzadi and R. Safabakhsh, "Text detection in natural scenes using fully convolutional densenets," in *Proc. 4th Iran. Conf. Signal Process. Intell. Syst. (ICSPIS)*, Dec. 2018, pp. 11–14.

[25] N. Takahashi and Y. Mitsufuji, "Multi-scale multi-band densenets for audio source separation," in *Proc. IEEE Workshop Appl. Signal Process. Audio Acoust. (WASPAA)*, Oct. 2017, pp. 21–25.

[26] D. M. Hutton, "The cross-entropy method: A unified approach to combinatorial optimisation, Monte-Carlo simulation and machine learning," *Kybernetes*, vol. 34, no. 6, p. 903, 2005.

[27] D. S. Emmerich, "Receiver operating characteristics determined under several interaural conditions of listening," *J. Acoust. Soc. America*, vol. 43, no. 2, pp. 298–307, 1968.

[28] M. Rasouli, A. Okhovatian, and A. Enderami, "Serum proteins profile as an indicator of malignancy: Multivariate logistic regression and ROC analyses," *Clin. Chem. Lab. Med. CCLM/FESCC*, vol. 43, pp. 913–918, Sep. 2005.

[29] L. C Brumback, M. S Pepe, and T. A Alonzo, "Using the ROC curve for gauging treatment effect in clinical trials," *Stat. Med.*, vol. 25, pp. 575–590, Feb. 2006.

[30] J. L. Raheja, B. Ajay, and A. Chaudhary, "Real time fabric defect detection system on an embedded DSP platform," *Optik*, vol. 124, no. 21, pp. 5280–5284, 2013.

[31] C.-H. Chan and G. K. Pang, "Fabric defect detection by Fourier analysis," *IEEE Trans. Ind. Appl.*, vol. 36, no. 5, pp. 1267–1276, Sep./Oct. 2000.

[32] S. C. Kim and T. J. Kang, "Texture classification and segmentation using wavelet packet frame and Gaussian mixture model," *Pattern Recognit.*, vol. 40, no. 4, pp. 1207–1221, 2007.

[33] J. P. Monaco and A. Madabhushi, "Class-specific weighting for Markov random field estimation: Application to medical image segmentation," *Med. Image Anal.*, vol. 16, no. 8, pp. 1477–1489, 2012.

[34] C.-Y. Lin, C.-H. Chen, C.-Y. Yang, F. Akhyar, C.-Y. Hsu, and H.-F. Ng, "Cascading convolutional neural network for steel surface defect detection," in *Proc. Int. Conf. Appl. Human Factors Ergonomics*, 2019, pp. 202–212.

[35] J.-F. Jing, H. Ma, and H.-H. Zhang, "Automatic fabric defect detection using a deep convolutional neural network," *Coloration Technol.*, vol. 135, no. 3, pp. 213–223, 2019.

[36] H. Lin, B. Li, X. Wang, Y. Shu, and S. Niu, "Automated defect inspection of led chip using deep convolutional neural network," *J. Intell. Manuf.*, vol. 30, no. 6, pp. 2525–2534, 2019.

[37] S. Zhang, X. Zhu, Z. Lei, H. Shi, X. Wang, and S. Z. Li, "S3FD: Single shot scale-invariant face detector," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 192–201.

[38] L. Perez and J. Wang, "The effectiveness of data augmentation in image classification using deep learning," 2017. [Online]. Available: https://arxiv.org/abs/1712.04621

[39] J. Pan and J. McElhannon, "Future edge cloud and edge computing for Internet of Things applications," *IEEE Internet Things J.*, vol. 5, no. 1, pp. 439–449, Feb. 2018.

[40] G. Premsankar, M. Di Francesco, and T. Taleb, "Edge computing for the Internet of Things: A case study," *IEEE Internet Things J.*, vol. 5, no. 2, pp. 1275–1284, Apr. 2018 .

[41] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proc. IEEE*, vol. 107, no. 8, pp. 1738–1762, Aug. 2019.

[42] K. Zhang, Y. Zhu, S. Maharjan, and Y. Zhang, "Edge intelligence and blockchain empowered 5G beyond for the industrial Internet of Things," *IEEE Netw.*, vol. 33, no. 5, pp. 12–19, Sep./Oct. 2019.

[43] E. Li, L. Zeng, Z. Zhou, and X. Chen, "Edge AI: On-demand accelerating deep neural network inference via edge computing," *IEEE Trans. Wireless Commun.*, vol. 19, no. 1, pp. 447–457, Jan. 2020.

[44] G. Liu, S. Liu, K. Muhammad, A. K. Sangaiah, and F. Doctor, "Object tracking in vary lighting conditions for fog based intelligent surveillance of public spaces," *IEEE Access*, vol. 6, pp. 29283–29296, 2018.

[45] Z. Piao, M. Peng, Y. Liu, and M. Daneshmand, "Recent advances of edge cache in radio access networks for Internet of Things: Techniques, performances, and challenges," *IEEE Internet Things J.*, vol. 6, no. 1, pp. 1010–1028, Feb. 2019.

[46] F. Samie, L. Bauer, and J. Henkel, "From cloud down to things: An overview of machine learning in Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4921–4934, Jun. 2019.



Zongwei Zhu received the M.S. and Ph.D. degrees in computer science from the University of Science and Technology of China (USTC), Hefei, China, in 2011 and 2014, respectively.

From 2014 to 2016, he was a Research Assistant with the IOT Perception Mine Research Center, China University of Mining and Technology, Xuzhou, China. From 2016 to 2018, he worked as a Senior Engineer with Huawei Company, Shenzhen, China. He is currently a Research Assistant with Suzhou Institute, USTC. His research focuses on edge computing, cloud computing, and power optimization.



Guangjie Han (Senior Member, IEEE) received the Ph.D. degree from Northeastern University, Shenyang, China, in 2004.

He is currently a Distinguished Professor with the College of Engineering, Nanjing Agricultural University, Nanjing, China, and a Professor with the Department of Information and Communication Systems, Hohai University, Changzhou, China. In February 2008, he finished his work as a Postdoctoral Researcher with the Department of Computer Science, Chonnam National University, Gwangju, South Korea. From October 2010 to October 2011, he was a Visiting Research Scholar with Osaka University, Suita, Japan. From January 2017 to February 2017, he was a Visiting Professor with the City University of Hong Kong, Hong Kong. He has authored over 380 papers published in related international conference proceedings and journals, including the IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE COMMUNICATIONS SURVEYS AND TUTORIALS, the IEEE TRANSACTIONS ON MOBILE COMPUTING, the IEEE TRANSACTIONS ON INDUSTRIAL ELECTRONICS, the IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, the IEEE TRANSACTIONS ON CLOUD COMPUTING, the IEEE TRANSACTIONS ON PARALLEL & DISTRIBUTED SYSTEMS, the IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, the IEEE INTERNET OF THINGS JOURNAL, IEEE SYSTEMS, IEEE WIRELESS COMMUNICATIONS, IEEE COMMUNICATIONS, and IEEE NETWORK. He is the holder of 130 patents. His current H-index is 40 and i10-index is 130 in Google Citation (Google Scholar). Total citation of his papers by other people is more than over 6300 times. His current research interests include Internet of Things, industrial Internet, machine learning and artificial intelligence, mobile computing, and security and privacy.

Prof. Han had been awarded the ComManTel 2014, ComComAP 2014, Chinacom 2014, Qshine 2016, and IEEE SYSTEMS JOURNAL 2020 Best Paper Awards. He has served as a Co-Chair for more than 50 international conferences/workshops and as a Technical Program Committee Member of more than 150 conferences. He has served on the editorial boards of up to 16 international journals, including the IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE NETWORK, IEEE SYSTEMS, IEEE ACCESS, the IEEE/CAA JOURNAL OF AUTOMATICA SINICA, and *Telecommunication Systems*. He has guest edited a number of special issues in IEEE journals and magazines, including IEEE COMMUNICATIONS, IEEE WIRELESS COMMUNICATIONS, the IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, and *Computer Networks*. He has served as a reviewer of more than 60 journals.

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 06:12:10 UTC from IEEE Xplore. Restrictions apply.



Gangyong Jia (Member, IEEE) received the Ph.D. degree from the Department of Computer Science, University of Science and Technology of China, Hefei, China, in 2013.

He is currently an Associate Professor with the Department of Computer Science, Hangzhou Dianzi University, Hangzhou, China. His current research interests are IoT, cloud computing, edge computing, power management, and operating system.

Dr. Jia has served as a reviewer for *Microprocessors and Microsystems*.



Lei Shu (Senior Member, IEEE) received the B.Sc. degree in computer science from the South Central University for Nationalities, Wuhan, China, in 2002, the M.Sc. degree in computer engineering from Kyung Hee University, Seoul, South Korea, in 2005, and the Ph.D. degree from the Digital Enterprise Research Institute, National University of Ireland, Galway, Ireland, in 2010.

Until 2012, he was a Specially Assigned Researcher with the Department of Multimedia Engineering, Graduate School of Information Science and Technology, Osaka University, Suita, Japan. He is currently a Distinguished Professor with Nanjing Agricultural University (NAU), Nanjing, China, and a Lincoln Professor with the University of Lincoln, Lincoln, U.K. He is also the Director of the Lincoln Joint Research Center of Intelligent Engineering, NAU. He has published over 400 papers in related conferences, journals, and books in the areas of sensor networks and Internet of Things. His current H-index is 48 and i10-index is 178 in Google Scholar Citations. His main research fields are wireless sensor networks and Internet of Things.

Dr. Shu was a recipient of the Top Level Talents in Sailing Plan of Guangdong Province, China, in 2014, the Outstanding Young Professor of Guangdong Province in 2015, and the GLOBECOM 2010, ICC 2013, ComManTel 2014, WICON 2016, and SigTelCom 2017 Best Paper Awards, the IEEE Systems Journal Best Paper Awards in 2017 and 2018, the *Journal of Network and Computer Applications* Best Research Paper Award in 2017, and the Outstanding Associate Editor Award of IEEE ACCESS in 2017 and 2018. He has also served over 50 various Co-Chair for international conferences/workshops, such as IWCMC, ICC, ISCC, ICNC, and Chinacom; the Symposium Co-Chair for IWCMC 2012 and ICC 2012; the General Co-Chair for Chinacom 2014, Qshine 2015, Collaboratecom 2017, DependSys 2018, and SCI 2019; the TPC Chair for InisCom 2015, NCCA 2015, WICON 2016, NCCA 2016, Chinacom 2017, InisCom 2017, WMNC 2017, and NCCA 2018; and a TPC Member of over 150 conferences, such as ICDCS, DCOSS, MASS, ICC, GLOBECOM, ICCCN, WCNC, and ISCC.

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 06:12:10 UTC from IEEE Xplore. Restrictions apply.